

Autonomous Travel-Disruption Concierge

Hackathon Playbook — American Express CodeStreet 2026

0. Read This First

The "Card Member" language in your brief, plus the exact track name, matches **American Express CodeStreet 2026** (run on HackerEarth) closely enough that this playbook is written directly for it. If that's not your competition, ignore the dates below — everything from Section 1 onward stands on its own.

Here's the timeline, as publicly listed:

Phase	Window	What's due
Registration & Idea Submission (Round 1)	Jul 13 – Jul 27, 2026	Project description + presentation — this document is built for that deadline
Prototype Submission & Mentoring (Round 2)	Aug 7 – Aug 21, 2026	A working prototype, for teams shortlisted out of Round 1
Grand Finale	Aug 25, 2026, in Chennai	Live pitch to Amex leadership; top 6 teams; top 2 get Pre-Placement Interview offers

That puts Round 1 within about a day of you reading this — the listed cutoff works out to roughly 11:59 PM IST on July 27, but **confirm the exact time on your HackerEarth dashboard**, don't trust my arithmetic on something this important.

Given that, this playbook is split in two:

- **Part A (Sections 1-14)** — everything needed to ship a strong Round 1 submission *tonight*. This is where your time should go right now.
- **Part B (Sections 15-20)** — the deeper build plan for the Aug 7-21 prototype window, if you're shortlisted. Come back for it in August; don't spend tonight's hours here.

If you have under two hours: read Section 2, lift Section 1 as-is, and use Section 12 to build your deck. **If you have an evening:** also work through Sections 5-9 and attempt Section 13.

PART A — Shipping Round 1 (due ~Jul 27)

1. Executive Summary

(Drop this into your project description field, then adjust to your voice.)

Close to one in four U.S. flights this year departed late or didn't depart at all. When that happens mid-itinerary, the Card Member is the one left standing at a gate, re-booking their

own connection, re-negotiating their own hotel, and often absorbing costs their card may already entitle them to avoid. Existing travel benefits are reactive — they reimburse a disruption after it has already ruined the trip. Airlines do eventually rebook affected passengers, but slowly, inconsistently, and never touching the hotel side of the disruption at all.

The **Autonomous Travel-Disruption Concierge** closes that gap. It continuously checks every active itinerary against live flight status and delay predictions, and the moment a disruption threatens an itinerary — or, where a delay is predicted in advance, *before* the moment — it independently searches, scores, and books a replacement flight and hotel stay, inside clearly defined policy limits, and only interrupts the Card Member when a decision genuinely needs a human call. Every autonomous action is logged, explainable, and reversible within a short window, so "autonomous" never becomes "unaccountable."

The result isn't a card benefit that pays for a ruined trip after the fact. It's one that stops the trip from being ruined.

2. Winning Strategy — What These Judges Actually Score

Publicly available guidance about this track's Round 1 evaluation points to four things: **implementation feasibility, business impact, clarity of problem understanding, and how honestly you handle trade-offs** — not how many AI buzzwords fill a slide. That maps directly onto four traps most teams will fall into:

Common trap	What this playbook does instead
A slide of API logos connected by arrows	A named, justified architecture using real Amadeus endpoints, with a stated reason for every component (Section 5)
"The AI agent decides everything"	A 3-tier autonomy model (Section 8) — the clearest signal to a financial-services judge that you understand you're touching someone's money and travel plans, not just calling an LLM in a loop
A generic itinerary-planner demo	A live "inject a disruption" trigger (Section 13/17), so judges watch detection → decision → booking → notification happen, instead of reading about it
Vague claims of "fast" and "reliable"	The brief names three metrics explicitly — detection speed, rebooking success rate, notification clarity. Section 11 shows exactly how to produce real numbers for each one

If the official rules publish a scoring rubric anywhere on the HackerEarth page, follow that exactly over this generic framework — this is informed inference, not the rulebook.

3. The Problem, With Numbers

Government aviation data for the year through mid-2025 shows nearly one in four U.S. flights arriving late or not operating at all, with disruption rates above a quarter of all flights in the worst-affected states. System-wide on-time performance has been sliding rather than improving: 2024's on-time arrival rate came in at 77.9%, the second-worst mark in recent years, and roughly 1.7 million of the 7.5 million flights scheduled by the ten largest U.S. airlines and their partners were delayed or cancelled that year.

For the traveler, a disruption rarely ends with the flight itself — it cascades into missed connections, hotel check-ins that no longer line up with a new arrival time, and hundreds of dollars in unplanned out-of-pocket costs, usually absorbed while standing at a gate with a phone in hand.

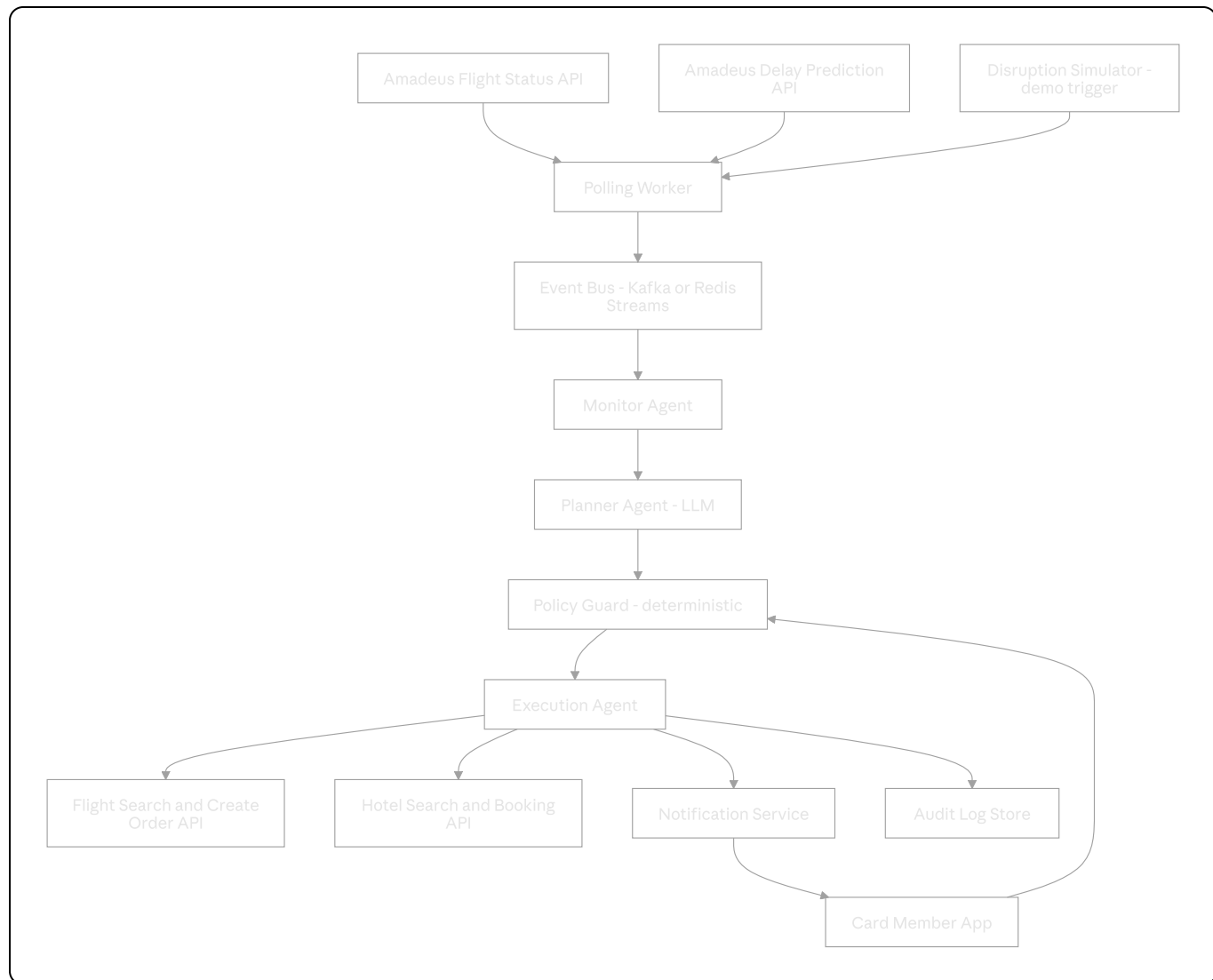
None of this is a capacity problem a card issuer can solve. But the *experience* around it is exactly what a card benefit can own — because the real gap isn't the disruption, it's the seam between what the airline eventually handles and what the Card Member is left to handle alone in the meantime.

4. The Core Idea — Three Pillars

One-line pitch: *Not an app that shows you rebooking options. A concierge that already booked the good one.*

1. **Detect early, not late** — combine reactive monitoring (live status changes) with predictive monitoring (delay-probability scoring), so alternatives are pre-computed for at-risk itineraries before a disruption is even confirmed. This directly optimizes the "detection speed" metric the brief calls out.
2. **Decide safely, not just cleverly** — an LLM proposes; a deterministic, auditable policy engine disposes. Three tiers of autonomy mean "autonomous" never means "unaccountable" (Section 8).
3. **Act completely, not partially** — flight, hotel, and notification move together as one resolution, with a full audit trail and a short undo window, so the Card Member experiences one seamless fix instead of three separate problems to solve themselves.

5. System Architecture



Component	Responsibility
Polling / ingestion workers	Poll Amadeus On-Demand Flight Status and Flight Delay Prediction per active segment; push normalized events onto the event bus
Event bus	Decouples detection from decision-making, and makes every event replayable — this is exactly what makes your demo controllable in front of judges
Monitor Agent	Classifies each event (cancelled, delayed, diverted, at-risk connection) and computes a risk score
Planner Agent (LLM)	Given a confirmed disruption, searches Flight Offers Search / Hotel Search for candidates and reasons through the trade-offs an LLM is genuinely good at — e.g. "this PNR has a tight layover history, prefer fewer connections"
Policy Guard	Deterministic, non-LLM code that checks every candidate action against a policy config before anything is allowed to execute (see Section 8)
Execution Agent	Calls Flight Create Orders / Hotel Booking, using idempotency keys so a retry can never double-book
Notification Service	Multi-channel delivery (push, SMS, email, WhatsApp) with a plain-language "why" explanation attached
Card Member App	Live alert feed, one-tap approve/override, audit trail view
Audit log store	Immutable record of every decision, its inputs, and its outcome

6. Algorithm Design

6.1 Disruption Detection

```
python
```

```

def evaluate_segment(segment, live_status):
    event = classify_event(live_status) # CANCELLED, DELAYED, DIVERTED, ON_TIME
    if event == "ON_TIME":
        return None

    disruption = DisruptionEvent(
        pnr=segment.pnr,
        segment_id=segment.id,
        event_type=event,
        new_eta=live_status.eta,
        delay_minutes=live_status.delay_minutes,
        detected_at=now(),
    )

    if segment.has_connection:
        buffer_minutes = minutes_between(live_status.eta, segment.next_segment.eta)
        mct = get_minimum_connect_time(segment.arrival_airport, segment.next_segment.arrival_airport)
        disruption.connection_risk = risk_tier(buffer_minutes, mct)

    return disruption

def risk_tier(buffer_minutes, mct_minutes, safety_margin=15):
    if buffer_minutes < mct_minutes:
        return "MISSED"
    if buffer_minutes < mct_minutes + safety_margin:
        return "AT_RISK"
    return "SAFE"

```

Feed the Flight Delay Prediction API's probability score in *before* a cancellation is confirmed: once a monitored flight crosses a delay-probability threshold, pre-compute the top three rebooking candidates and hold them in cache. If the disruption then confirms, execution starts from a warm cache instead of a cold search — this is the single biggest lever for the "detection speed" metric.

6.2 Rebooking Decision

python

```

def score_candidate(candidate, original, policy):
    delay_penalty = -abs(minutes_between(candidate.arrival, original.arrival))
    stops_penalty = -candidate.num_stops * 15
    cabin_bonus = 20 if candidate.cabin >= original.cabin else -50
    fare_delta = candidate.fare - original.fare
    fare_penalty = -fare_delta * 0.5 if fare_delta > 0 else 0
    alliance_bonus = 10 if candidate.alliance == original.alliance else 0
    return delay_penalty + stops_penalty + cabin_bonus + fare_penalty + alliance_bonus

def decide(candidates, original, policy):
    ranked = sorted(candidates, key=lambda c: score_candidate(c, original, policy))
    best = ranked[0]
    fare_delta = best.fare - original.fare

    if fare_delta <= policy.tier_1_cap and best.cabin >= original.cabin:
        return Decision(tier=1, action="AUTO_BOOK", option=best)
    elif fare_delta <= policy.tier_2_cap:
        return Decision(tier=2, action="CONFIRM_WITH_MEMBER", options=ranked[:2])
    else:
        return Decision(tier=3, action="ESCALATE_HUMAN", options=ranked[:3])

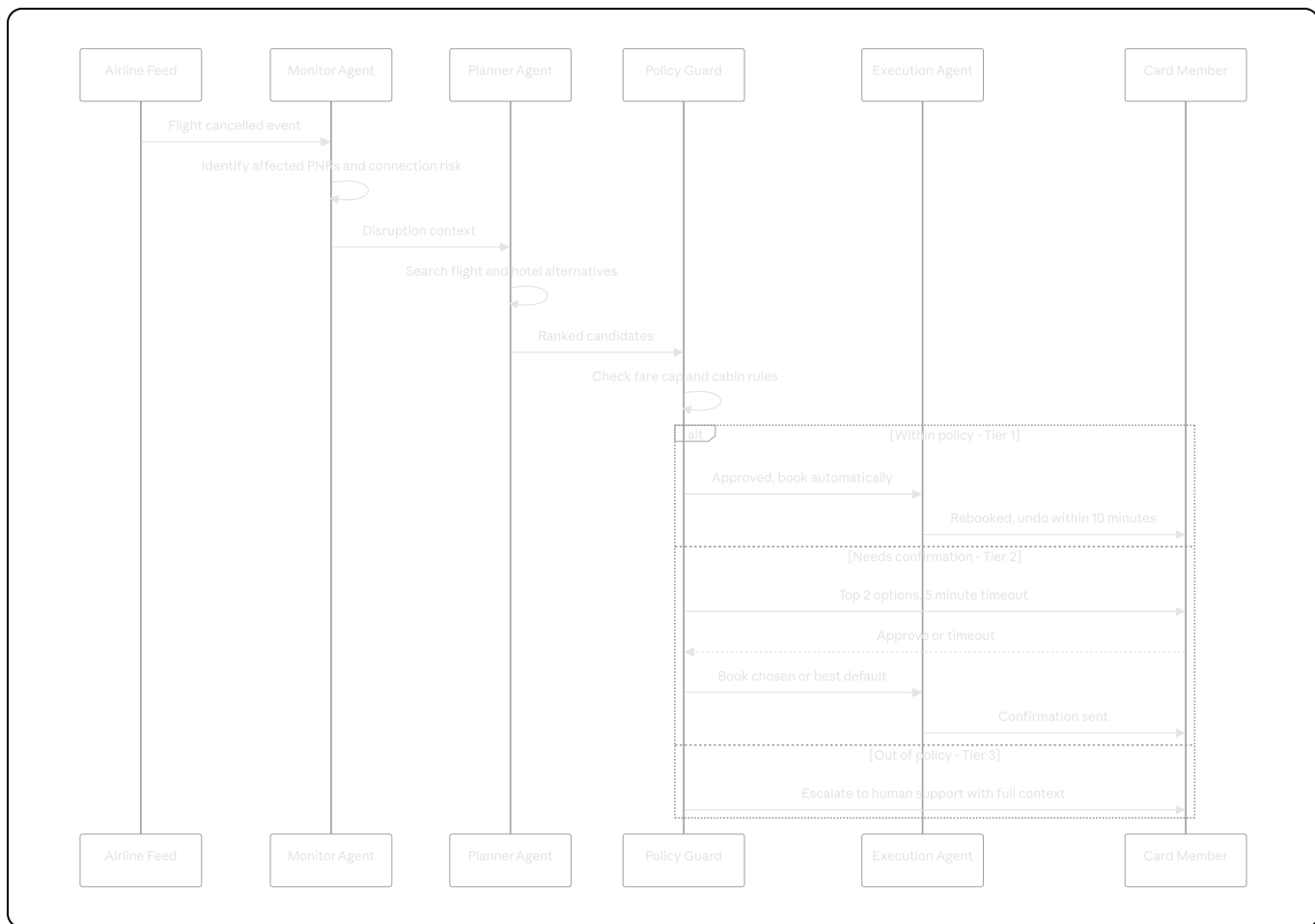
```

6.3 Hotel Rearrangement

Trigger condition: the new arrival requires an overnight stay not in the original plan, or the original hotel's check-in window no longer matches the revised itinerary. Before auto-executing a change, compute the *net* cost impact (new booking cost plus any cancellation fee on the original, checked against the original's cancellation deadline) — treat this exactly like a flight fare delta against the same policy caps.

6.4 Notification

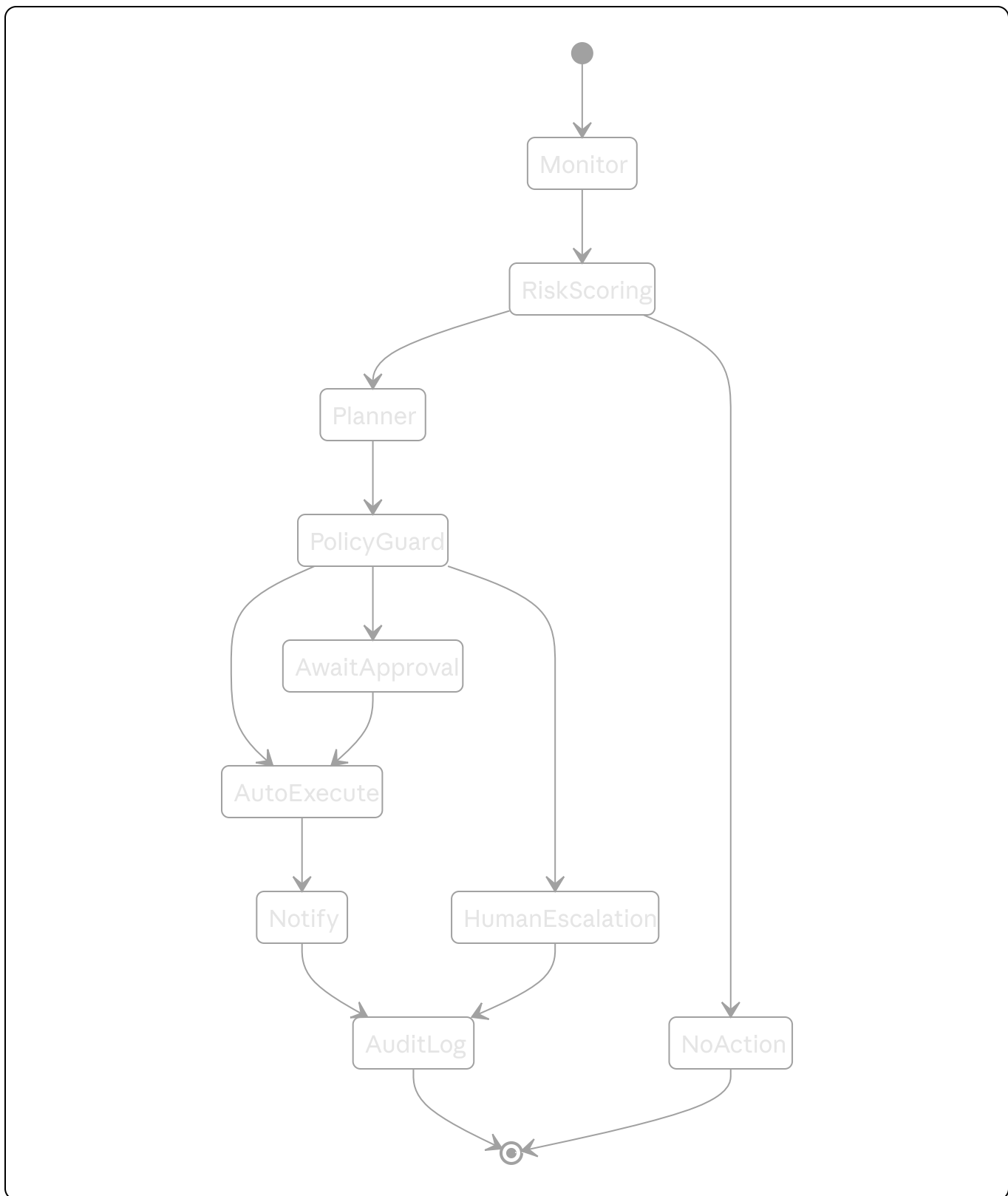
Every notification should answer three questions in under two lines: what changed, what was done about it, and how to undo it (if still within the undo window). That "why" line is what separates a trustworthy autonomous action from an alarming one — and it's free differentiation, since most teams will only show a bare confirmation message.



7. Agentic AI Design (LangGraph)

Model the whole flow as a LangGraph state graph rather than a single prompt loop — this is worth stating explicitly in your proposal, because it's a concrete, checkable design decision rather than an "AI agent" hand-wave.

The mechanism worth naming: LangGraph's `interrupt()` function pauses a running graph at any node and persists its state via a checkpoint; execution resumes later — seconds or days later — with `Command(resume=...)`, keyed by a `thread_id` per disruption. That's exactly the primitive Tier 2 and Tier 3 need: the graph genuinely pauses and waits for the Card Member (or a human support agent) rather than looping or polling. It's the same pattern used for approval workflows and financial-transaction oversight in production agent systems today, which is a reassuring thing to cite to judges who will be wondering "what stops this from doing something reckless."



Keep the Policy Guard node as plain code, not an LLM call. This is the single most important engineering decision in the whole system, and worth a full sentence in your proposal: probabilistic reasoning is excellent at *proposing* a good rebooking; it is the wrong tool for *enforcing* a hard dollar cap, and judges in financial services will notice if you conflate the two.

8. Trust, Guardrails & the 3-Tier Autonomy Model

This is very likely your strongest differentiator, because most teams will treat "autonomous" as the whole pitch and stop there. A three-tier autonomy model resolves the tension between the brief's demand for "no manual action required" and the reality that some decisions shouldn't be fully automatic:

yaml

```
autonomy_policy:
  tier_1_auto_book:
    description: "Minor, in-policy disruptions – booked immediately, no wait"
    max_fare_delta_pct: 15
    max_fare_delta_abs_usd: 200
    cabin_downgrade_allowed: false
    max_added_stops: 1
    undo_window_minutes: 10

  tier_2_confirm_first:
    description: "Moderate impact – proposed to the Card Member, auto-confirms"
    max_fare_delta_pct: 40
    max_fare_delta_abs_usd: 600
    response_timeout_minutes: 5
    default_on_timeout: best_ranked_option # still satisfies "no manual action"

  tier_3_escalate_human:
    description: "Complex, high-cost, or multi-passenger cases"
    condition: "fare_delta exceeds tier 2 OR international reroute OR multi-passenger"
    route_to: human_travel_desk
```

Three more pieces complete the trust story, and each is cheap to build but expensive-looking to judges:

- **Immutable audit trail** — every decision logged with the candidates considered, the scores, which policy checks passed or failed, and the final choice. This is what makes an autonomous system explainable after the fact, not just fast.
- **Undo window** — Tier 1 holds a booking before final ticketing for a short window, or leans on the airline's standard 24-hour risk-free cancellation rule where it applies to the fare. Naming a real regulatory hook here (this is a genuine, stable DOT rule, not something you're inventing) shows the trade-off awareness judges are reportedly looking for.
- **Cost governance** — log running spend per disruption resolved (API calls + fare deltas), so you can show a real cost-per-resolution number rather than an unbounded

"the agent just books things."

9. Tech Stack

Layer	Recommended	Why
Mobile / web app	React Native + a lightweight web dashboard	Cross-platform, fast to scaffold, native push support
Backend API	FastAPI (Python)	Async-friendly; shares a language with the agent layer
Agent orchestration	LangGraph	Explicit state machine with built-in human-in-the-loop <code>((interrupt)(Command(resume=...)))</code> — easier to defend to judges than a black-box agent loop
LLM	Any strong tool-calling model	Used only in the Planner node for reasoning over trade-offs, never for policy enforcement
Flight & hotel data	Amadeus Self-Service APIs (test environment)	Free sandbox keys, real schemas, no GDS contract needed for a hackathon
Event bus	Kafka, or Redis Streams if you want less infra to stand up	You've already covered Kafka's event-streaming model in CS-703, so it's a natural fit if you want the concept to click fast; Redis Streams is the lighter option if time is short
Database	PostgreSQL	Relational integrity for itineraries, PNRs, and the audit trail
Cache	Redis	Fast lookups for pre-computed, in-flight risk scores
Notifications	Firebase Cloud Messaging, Twilio (SMS), SendGrid (email), optionally WhatsApp Business API	Multi-channel reach; WhatsApp is a nice, locally-relevant touch
Cloud	AWS (Lambda/ECS + EventBridge + RDS)	Matches the brief's example stack, fits your existing AWS case-study background, and event-driven serverless suits disruption-triggered workloads well

10. Card-Member Experience

Four screens carry the whole story — don't build more than this for Round 1:

1. **Live alert** — "Your connection in Chicago is at risk. We're finding alternatives now."
(shown the instant risk is detected, before a decision is even made — this alone demonstrates the "detect early" pillar)
2. **Resolution summary** — what changed, what was booked, the one-line "why," and an undo button if still in-window
3. **Approve / override** (Tier 2 only) — top 2 options side by side, countdown to auto-confirm
4. **Audit trail** — a simple timeline of every step the agent took for this disruption, timestamped

11. Proving the Three Metrics the Brief Names

The brief explicitly asks you to optimize for "disruption detection speed, rebooking success rate, and notification clarity" — most teams will gloss this as a vague claim of quality. A submission that shows real numbers for exactly these three visibly outperforms one that just asserts "it works":

- **Detection speed** — log the timestamp of the source status change vs. the timestamp your Monitor Agent classified it; report it as a distribution (median/p95), not a single cherry-picked number.
- **Rebooking success rate** — % of triggered disruptions that reached a booked resolution without manual intervention, out of however many you can run through the simulator (Section 17) before submission.
- **Notification clarity** — this one's softer; a simple readability pass (does the message answer "what changed / what did you do / how do I undo it" in under two lines?) plus 2-3 informal test reads from people outside your team is enough to cite honestly.

12. Presentation Deck Outline

1. **Title** — solution name + one-line pitch from Section 4
2. **The hook** — a 15-second relatable scenario: a missed connection, a cancelled flight, a Card Member stuck re-booking themselves at 11pm
3. **The gap** — the numbers from Section 3, and the "airlines eventually rebook, but slowly and never touch the hotel" framing
4. **The solution** — the three pillars (Section 4)
5. **Architecture** — the Section 5 diagram, walked through in under a minute
6. **The trust layer** — the 3-tier autonomy model (Section 8); spend real time here, this is your differentiator
7. **Live or recorded demo** — the disruption-simulator trigger in action (Section 13/17)

8. **Metrics** — real numbers against the brief's three named metrics (Section 11)
9. **Business impact** — this turns a reactive insurance-style benefit into a proactive service benefit: fewer support-line calls during irregular operations, a stronger differentiator against competing travel cards, and a concrete trust story for a card that already touches the Card Member's money
10. **Roadmap** — what Round 2 (Aug 7-21) would add
11. **Close** — restate the one-liner

13. Tonight's Minimal Proof-of-Concept (optional, but a real differentiator)

A working link — even a small one — will stand out heavily among proposals that are slides only. Amadeus's test environment is free and instant, so this is realistic in a handful of hours solo:

- One small FastAPI service with three endpoints: `POST /simulate-disruption` (injects a fake cancelled-flight event for a demo PNR), `GET /status/{pnr}`, and a background worker that on receiving the event calls Amadeus Flight Offers Search (test environment), runs the Section 6.2 scoring function, books a real *test* order via Flight Create Orders, and logs a decision + notification message.
- Screen-record hitting `/simulate-disruption` and watching detection → scoring → booking → notification happen in the console within seconds. Even a terminal-only clip is a real, working artifact — far more convincing than a static deck.
- **If you truly have no spare hours:** at minimum, record a 60-second narrated walkthrough of the Section 5 architecture diagram, talking through the flow out loud. A narrated diagram beats a silent PDF.

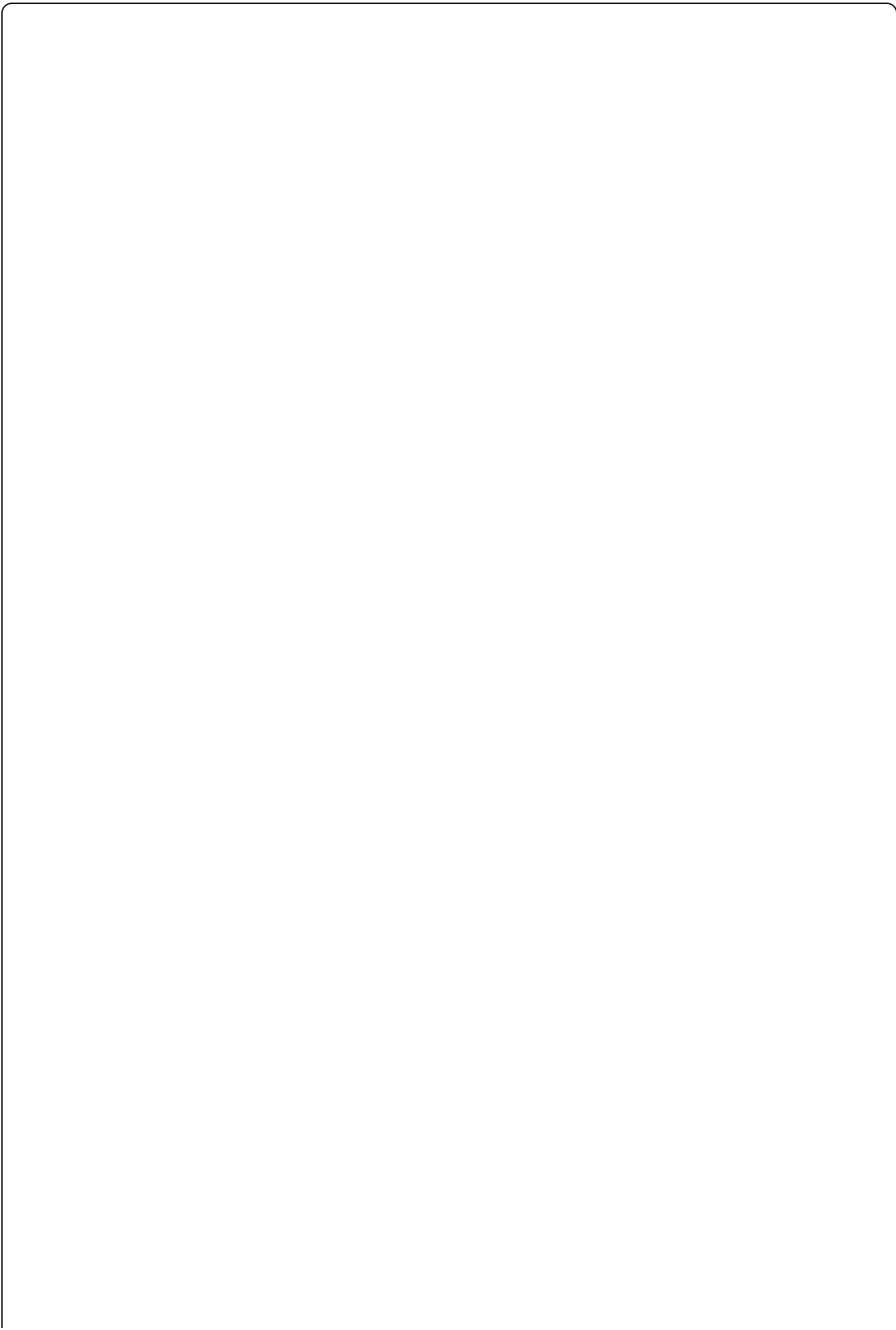
14. Round 1 Submission Checklist

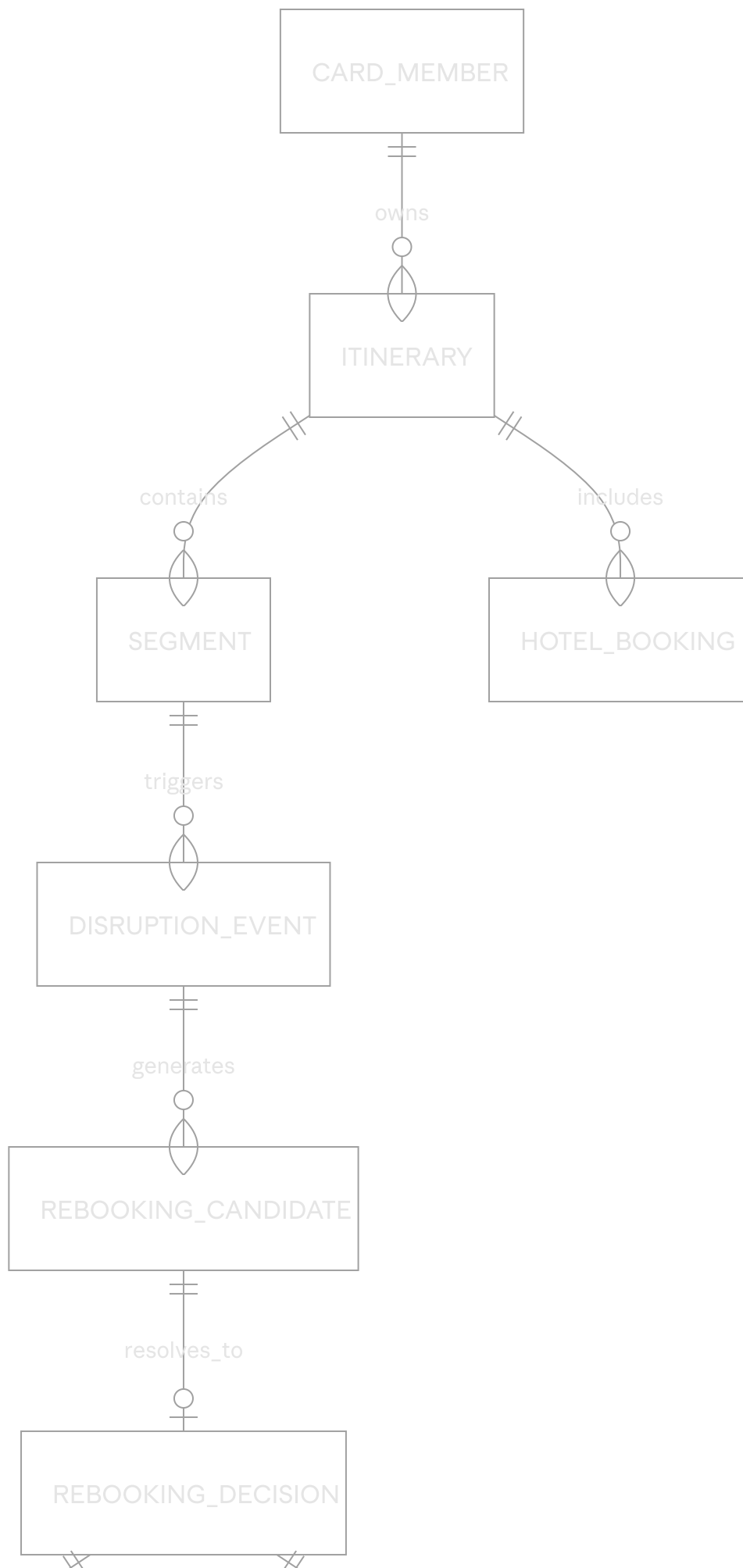
- ☐ Project description (Section 1, adapted to your voice)
- ☐ Presentation deck (Section 12 outline, built out)
- ☐ Supporting documentation — this playbook's Sections 5-9 cover the technical depth
- ☐ Video and link — even the minimal proof-of-concept from Section 13, or a narrated diagram walkthrough at minimum
- ☐ Confirm the exact Round 1 cutoff time on your HackerEarth dashboard

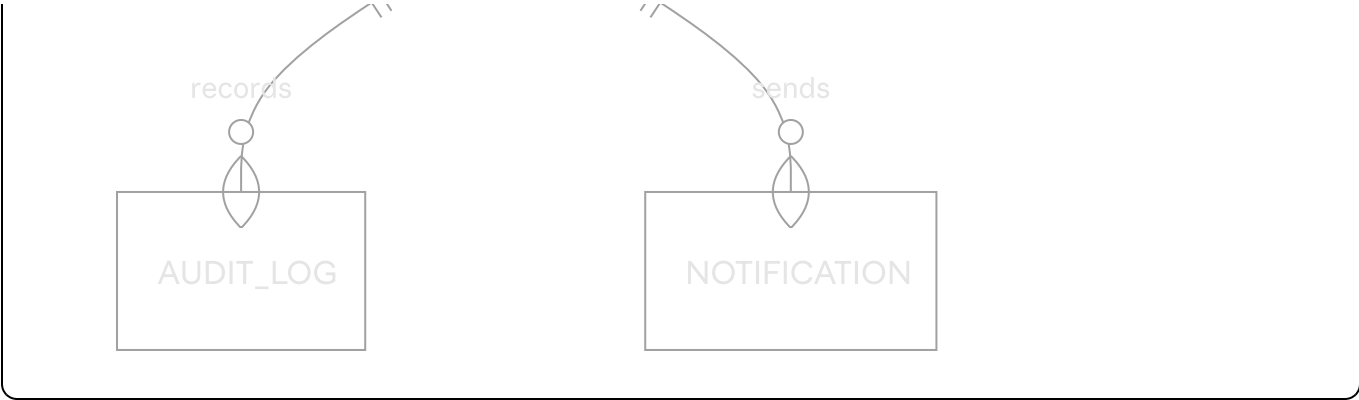
PART B — If You're Shortlisted (Prototype Phase, Aug 7-21 → Chennai Finale Aug 25)

Come back to this section in August. Don't spend tonight's hours here.

15. Data Model







Key tables: `card_members`, `itineraries`, `segments`, `disruption_events`, `rebooking_candidates`, `rebooking_decisions`, `hotel_bookings`, `policy_rules`, `notifications`, `audit_log`. Keep `audit_log` append-only — never update or delete rows — since its entire value is being an incorruptible record of what an autonomous system did and why.

16. Full Build Roadmap (Aug 7-21)

Work in percentage-of-time phases so this scales to however many days you actually get:

Phase	~% of time	Goal
Setup	5%	Amadeus sandbox keys, repo scaffolding, task split
Thin vertical slice	25%	Ingestion → detection → simple rule-based decision → mock execution → console notification, wired end to end even if ugly
Real intelligence	25%	Real Amadeus calls, LangGraph agent, the Section 6 scoring algorithm, the Section 8 policy guard and tiers
Card Member experience	20%	The four screens from Section 10, push notifications, approve/override flow
Demo hardening	15%	Build the disruption simulator (Section 17) properly, seed realistic demo data, rehearse
Presentation & video	10%	Finale deck, script, recording

Get the thin end-to-end slice working *first*, before polishing anything — a working demo of the full loop beats several disconnected, individually-polished features every time.

Team split (1-3 people): solo, prioritize the vertical slice and the trust-layer story over UI polish — judges reportedly weight feasibility and architecture over visual finish. With 2, split backend/agent vs. frontend/notifications. With 3, add a dedicated owner for the

Amadeus integration, demo data, and the simulator — that role ends up mattering more than it sounds like it should, because it's what makes your live demo actually work under pressure.

17. The Demo Trick — Simulating Disruptions on Command

The single most-overlooked practical problem: you cannot wait for a real flight to get cancelled during a live judging session. Build a controllable trigger as a first-class feature, not an afterthought — an "inject disruption" endpoint or a small admin panel that lets you (or a judge) fire a specific event against a seeded demo PNR on command. This turns "detects live disruptions" from a claim into something the judges watch happen in real time, which is a meaningfully different experience for them than a recorded video.

18. Finale Demo Video Script (3 minutes)

- **0:00-0:15** — Hook: the relatable scenario from Section 12, slide 2
- **0:15-0:45** — Trigger a disruption live; show the agent detecting it in real time
- **0:45-1:30** — Show the decision flow: candidates found, scored, booked; hotel adjusted
- **1:30-2:00** — Notification + the "why" explanation + audit trail
- **2:00-2:30** — Quick architecture flash + the trust/autonomy-tier story
- **2:30-2:45** — Business impact in one sentence
- **2:45-3:00** — Close on the one-liner

19. Anticipated Judge Questions

Question	Strong answer
"Airlines already rebook disrupted passengers — isn't this redundant?"	Eventually, yes, but slowly, inconsistently, and never touching the hotel side. This is the orchestration and speed layer across systems the airline itself doesn't unify.
"What if the autonomous pick is worse for the Card Member?"	That's exactly what the 3-tier model, the audit trail, and the undo window are for — walk through Section 8.
"How would this get real airline system access in production?"	Through GDS partners like Amadeus/Sabre and NDC integrations, the same route existing travel-booking card benefits already use.
"Who pays the fare difference?"	The policy caps in Section 8 — this plugs naturally into the trip-delay/cancellation-insurance-style benefit many premium cards already offer, turning a reactive payout into a proactive service.
"What about PNR and payment data security?"	Least-privilege API scopes, tokenized payment handling, no plaintext PII in logs — worth one slide if asked.

20. Stretch Goals

- Multilingual notifications
- A "why this and not that" comparison view showing the runner-up candidates the agent rejected, not just the winner
- A cost-governance dashboard showing spend per disruption resolved
- EU261-style compliance awareness for international itineraries, if time allows

Appendix — Sources & References

- [Amadeus for Developers — Flight Status API](#)
- [Amadeus for Developers — Automatically booking new flights based on likely delays](#)
- [Amadeus for Developers — Hotel Search API](#)
- [LangChain — Making it easier to build human-in-the-loop agents with interrupt](#)
- [U.S. PIRG — Plane Truth 2025](#)
- [Squaremouth — U.S. Flight Delay Statistics by State](#)
- [HackerEarth — CodeStreet 2026](#)
- [LangChain Documentation](#)